Reporte

Nombre:	**Matrícula**:
Jorge Antonio Granados de Luca Guzman Ibarra Edgar Leonardo Denilson Misael González Marquez	2151253 2148230 2148813
Nombre del curso: Componentes de lógica de negocios	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 2	**Actividad**: Diseño Arquitectural (Fase 2)
Fecha: 15/10/2026	
Bibliografia (Formato APA)	

Objetivo:

El presente documento tiene como objetivo principal organizar de manera exhaustiva las reglas, responsabilidades y estructuras internas del sistema FitZone antes de proceder a su codificación completa. A través de este informe, el equipo de desarrollo demuestra la transición lógica y técnica desde los requerimientos de negocio levantados en la Unidad 2 (BRD, BPMN, Historias de Usuario) hacia una arquitectura de software robusta, escalable y mantenible. Se justifica detalladamente la separación de responsabilidades utilizando Domain-Driven Design (DDD), separando el dominio de la aplicación y la infraestructura, implementando patrones como Domain Model, Service Layer, CQRS, Repository y Unit of Work, y validando todo el diseño bajo los principios SOLID.

Reporte

Procedimiento :

1. Etapa 1: Preparación del Caso y Análisis de Casos de Uso

Para el diseño arquitectural se han seleccionado cuatro casos de uso críticos que abarcan los flujos principales y alternativos más complejos del negocio. Cada caso de uso interactúa directamente con los Aggregate Roots definidos en el modelo de dominio.

CU-01: Identificación y Acceso en Recepción

Actor: Socio / Recepción

Descripción: El sistema debe validar en tiempo real si un socio tiene permitido el acceso físico a las instalaciones del gimnasio.
Precondiciones: El socio debe estar registrado en el sistema y presentar su identificador (ID/Tarjeta).
Reglas de Negocio (Invariantes): La membresía debe estar en estado 'Activa'. No debe tener estatus de 'Suspendida'. El acceso debe registrarse dentro del horario permitido por su Plan Contratado.
Flujo Principal: El socio escanea su ID -> El sistema recupera el Aggregate 'Membresia' -> Se invoca el método validarVigencia() -> El sistema emite un evento de 'AccesoPermitido' -> Se guarda el registro en el historial.
Flujo Alternativo: Si la membresía está vencida, el sistema arroja alerta roja, emite 'AccesoDenegado' y bloquea el torniquete.

CU-02: Reservación de Clase Específica

Actor: Socio

Descripción: Permite a un socio asegurar un lugar en una clase especial sujeta a restricciones de capacidad.
Precondiciones: Membresía activa y plan que incluya la clase deseada.
Reglas de Negocio (Invariantes): La 'Capacidad Máxima' de la clase no puede ser excedida bajo ninguna circunstancia. El socio no puede reservar la misma clase dos veces en el mismo horario.
Flujo Principal: El socio selecciona la clase -> El sistema valida inclusiones del Plan -> El sistema consulta los cupos disponibles en el Aggregate 'ClaseEspecial' -> Se crea la 'Reserva' -> Se reduce el cupo -> Se emite 'ReservaConfirmada'.
Flujo Alternativo (Rechazo): Si el plan no incluye la clase, se detiene el flujo y se solicita un pago adicional (CU-04).

CU-03: Ingreso y Gestión de Lista de Espera

Actor: Sistema / Socio

Descripción: Mecanismo de contingencia cuando una clase especial alcanza su límite físico de asistentes.
Precondiciones: La 'ClaseEspecial' debe tener sus cuposDisponibles en 0.
Reglas de Negocio (Invariantes): El orden de la lista es estrictamente secuencial (FIFO). Solo se asigna un lugar cuando ocurre un evento de 'LugarLiberado'.
Flujo Principal: El socio intenta reservar -> El sistema detecta cupo lleno -> El socio es agregado como entidad 'ListaEspera' vinculada a la clase -> Se emite 'CapacidadAlcanzada'.

CU-04: Cobro de Clases Fuera de Plan

Actor: Recepción

Descripción: Gestión de pagos excepcionales para permitir accesos a servicios no incluidos en el plan base.
Precondiciones: Un socio intenta acceder o reservar un servicio premium no cubierto por su suscripción actual.
Reglas de Negocio (Invariantes): El pago debe ser procesado y confirmado por el sistema bancario antes de emitir la autorización temporal.
Flujo Principal: Recepción inicia el cobro -> Se registra la transacción -> Se genera el objeto 'Pago' -> Se asocia el pago a la 'Reserva' o 'Acceso' -> Se habilita el servicio.

2. Etapa 2: Elección del Patrón de Lógica de Negocio

La arquitectura central del sistema define cómo se procesan, organizan y mantienen las reglas de negocio. Para FitZone, se analizaron tres patrones fundamentales propuestos por Martin Fowler: Transaction Script, Table Module y Domain Model.

2.1 Análisis Comparativo

Criterio Arquitectural	Transaction Script	Table Module	Domain Model
Manejo de Complejidad de Reglas	Agrupa la lógica por acción (ej. Procedimiento ReservarClase). Tiende a duplicar validaciones si otra acción requiere las mismas reglas.	Organiza la lógica alrededor de tablas de base de datos. Bueno para CRUD masivos, pésimo para reglas complejas inter-tablas.	Encapsula las reglas dentro de objetos de negocio enriquecidos. Las entidades son responsables de proteger su propio estado.
Resiliencia ante Cambios	Frágil. Un cambio en la política de membresías obliga a modificar múltiples scripts de transacción.	Requiere alterar los Recordsets y las clases que manejan las tablas directamente, afectando la capa de acceso a datos.	Altamente resiliente. Un cambio en las reglas de aforo solo modifica el Aggregate Root ClaseEspecial, sin tocar la base de datos.
Protección de Invariantes y Estado	Nula. Cualquier script puede modificar los datos directamente en la base de datos, saltándose reglas.	Media. Protege la tabla a nivel global, pero es difícil coordinar transacciones que afecten múltiples tablas a la vez.	Máxima. El Aggregate Root es el único punto de entrada para modificar los datos, garantizando que el sistema nunca quede en un estado inválido (ej. cupos negativos).
Alineación con el BRD (Evidencia U2)	No refleja el lenguaje ubicuo del BRD, solo funciones imperativas.	Se acopla al modelo Entidad-Relación, no al negocio.	Mapea directamente el Glosario de la Fase 1 a objetos reales (Socio, Membresia, Reserva).

Curva de Aprendizaje y Mantenimiento	Fácil de implementar al inicio, insostenible a largo plazo (Código Espagueti).	Intermedia, requiere frameworks específicos de manejo de Recordsets.	Alta al inicio (requiere DDD), pero garantiza un mantenimiento predecible a largo plazo.

2.2 Justificación de la Decisión: Domain Model

Decisión Final: El equipo ha seleccionado unánimemente el patrón Domain Model.

Argumentación: El sistema FitZone no es un simple CRUD (Crear, Leer, Actualizar, Borrar). Contiene un entramado de reglas de negocio fuertemente acopladas entre sí. Por ejemplo, permitir que un socio reserve una clase (US-02) requiere verificar su identidad (Socio), comprobar las fechas de su suscripción (Vigencia), revisar si el tipo de suscripción cubre el servicio (Plan Contratado) y validar que el espacio físico del gimnasio lo permita (Capacidad Máxima).

Si utilizáramos Transaction Script, tendríamos que escribir sentencias condicionales masivas en cada controlador o servicio de la aplicación. Con Table Module, tendríamos problemas de acoplamiento al intentar coordinar la tabla de 'Socios' con la de 'Clases'. El Domain Model nos permite abstraer toda esta complejidad. Creamos un modelo de objetos enriquecido donde la clase 'Membresia' tiene el método 'validarVigencia()' y la clase 'ClaseEspecial' tiene el método 'reducirCupo()'. Esto centraliza el conocimiento del negocio, evita la duplicación de código y nos asegura que ninguna reserva se pueda guardar en la base de datos si viola las reglas dictadas en el BRD.

3. Etapa 3: Mapa de Dominio y Lenguaje Ubicuo

A continuación se desglosa la anatomía del modelo de dominio, separando los conceptos en Aggregate Roots (raíces de agregación que controlan transacciones), Entities (objetos con identidad propia), Value Objects (objetos inmutables definidos por sus atributos) y Domain Events (notificaciones asíncronas de negocio).

3.1 Aggregate Roots (Raíces de Agregación)

1. Aggregate Root: Socio
- Identidad: ID de Socio único (Matrícula/Número de cliente).
- Responsabilidad: Representar al individuo físico. Mantiene la lista de reservas históricas y gestiona su perfil.
- Invariantes: Un socio no puede existir en el sistema sin datos de contacto válidos.

2. Aggregate Root: Membresía
- Identidad: ID de Membresía.
- Responsabilidad: Actuar como el motor de permisos del sistema. Dictamina qué puede y qué no puede hacer el socio dentro de las instalaciones.
- Invariantes: Las fechas de inicio y fin deben ser lógicas (fin > inicio). Su estado se recalcula basado en la fecha del sistema.

3. Aggregate Root: ClaseEspecial
- Identidad: ID de Clase Programada (ej. 'Yoga-Lunes-10AM').
- Responsabilidad: Controlar estrictamente el aforo físico. Es la única clase autorizada para crear entidades 'Reserva' o añadir personas a la 'ListaEspera'.
- Invariantes: Cupos Disponibles >= 0. No se pueden procesar reservas si el estado de la clase es 'Cancelada'.

3.2 Entities (Entidades Internas)

• Reserva: Entidad transaccional. Tiene un ID propio, pero su ciclo de vida depende completamente de la ClaseEspecial. Si la clase se cancela, la reserva deja de tener sentido de negocio.

• Lista de Espera: Entidad secuencial. Administra posiciones (1, 2, 3...) y asegura el principio FIFO para reasignación de cupos.

• Instructor: Entidad que representa al empleado. Sus responsabilidades dentro de este contexto se limitan a visualizar aforos y ejecutar la cancelación de la sesión.

3.3 Value Objects (Objetos de Valor)

Los Value Objects carecen de identidad propia. Si dos objetos 'Vigencia' tienen las mismas fechas, son exactamente el mismo valor. Son inmutables; para cambiarlos, se debe instanciar uno nuevo.

• Vigencia: { FechaInicio: DateTime, FechaVencimiento: DateTime }

• PlanContratado: { Tipo: Enum(Mensual, Trimestral, Anual), ServiciosIncluidos: List<String> }

• CapacidadMaxima: { Limite: Int }. Garantiza que nunca se inicialice con números negativos o decimales.

3.4 Domain Events (Eventos de Dominio)

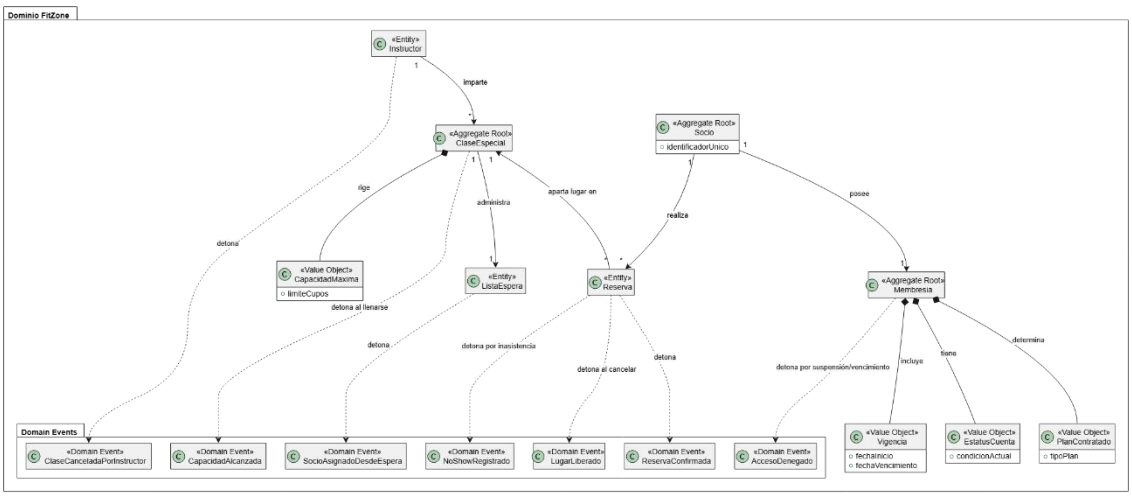
Los eventos permiten desacoplar procesos. Cuando un Aggregate hace algo importante, grita un evento para que otras partes del sistema reaccionen.

• LugarLiberado(ReservaId, ClaseId): Detonado cuando un socio cancela a tiempo. Su 'Listener' principal es el sistema de Lista de Espera.

• NoShowRegistrado(SocioId, ClaseId): Detonado por el sistema de recepción al finalizar la tolerancia. Provoca una penalización en el perfil del socio.

• AccesoDenegado(SocioId, Motivo): Utilizado para generar reportes analíticos de intentos de violación de acceso.x

MAPA DE DOMINIO



4. Etapa 4: Diseño Exhaustivo de la Service Layer

La Service Layer (Capa de Aplicación) actúa como una 'fachada'. Su única responsabilidad es recibir el comando del usuario (ej. de la API REST o la interfaz web), cargar los objetos de la base de datos (usando Repositories), decirle al Domain Model qué hacer, y guardar los cambios. NUNCA debe contener lógica de negocio (if cupos > 0).

Application Service	Entrada (DTO)	Colaboradores (Repos)	Flujo de Orquestación	Reglas que Delega al Dominio
ReservaAppService.ReservarClase()	socioId, claseId	SocioRepo, ClaseRepo, UnitOfWork	1. Obtiene Socio. 2. Obtiene Clase. 3. Llama a clase.AgregarReserva(socio). 4. UoW.Commit().	Delega a ClaseEspecial la verificación del aforo y a Membresia la vigencia.
AccesoAppService.ValidarIngreso()	socioId, horaActual	MembresiaRepo, AccesoRepo	1. Obtiene Membresía. 2. Llama a membresia.Autorizar(hora). 3. Guarda registro.	Delega la validación matemática de fechas y horarios permitidos.
AsistenciaAppService.MarcarNoShows()	claseId	ClaseRepo, SocioRepo	1. Obtiene Clase. 2. Filtra reservas sin asistencia. 3. Emite evento NoShow.	Delega la lógica de cálculo de tolerancias (ej. 10 minutos tarde = NoShow).

ListaEsperaAppService.AsignarVacante()	claseId	ClaseRepo, NotificacionService	1. Escucha evento LugarLiberado. 2. Saca primer socio de lista. 3. Notifica por email.	Delega el manejo de la cola y validación FIFO a la entidad ListaEspera.
MembresiaAppService.RenovarPlan()	socioId, planId, pagoId	MembresiaRepo, PagoExterno	1. Valida pago bancario. 2. Obtiene Membresía. 3. Llama a membresia.Extender().	Delega el cálculo matemático de la nueva fecha de vencimiento.

5. Etapa 5 y 6: Patrones de Persistencia, Transacciones y CQRS

Para mantener el principio de Inversión de Dependencias (D en SOLID), el dominio no sabe nada de SQL, Entity Framework ni bases de datos. Todo se abstrae en interfaces.

5.1 Interfaces de Repositorio (Repository Pattern)

```
public interface IClaseEspecialRepository {
    Task<ClaseEspecial> ObtenerConReservasAsync(int claseId);
    Task GuardarCambiosAsync(ClaseEspecial clase);
}

public interface IMembresiaRepository {
    Task<Membresia> ObtenerMembresiaActivaSocioAsync(int socioId);
}
```

5.2 Unit of Work (Patrón de Unidad de Trabajo)

El patrón Unit of Work garantiza la atomicidad de las operaciones (ACID). Si un servicio intenta reservar una clase y realizar un cobro, ambas acciones deben completarse juntas. Si el cobro falla por conexión al banco, el Unit of Work ejecuta un rollback automático, evitando que se asigne un lugar no pagado y asegurando la consistencia de los datos.

5.3 Command Query Responsibility Segregation (CQRS)

CQRS separa radicalmente las operaciones que alteran el estado del sistema (Commands) de las que solo devuelven datos (Queries).

Tipo / Nombre	Data Transfer Object (Entrada)	Handler (Procesador)	Beneficio Arquitectural
Command: ReservarClaseCommand	{ SocioId: Int, ClaseId: Int }	ReservarClaseCommandHandler	Encapsula la intención de mutación. Dispara eventos de dominio.
Command: CancelarClaseCommand	{ InstructorId: Int, ClaseId: Int, Motivo: String }	CancelarClaseCommandHandler	Cambia el estado masivamente y detona notificaciones a todos los socios.

Query: ConsultarAforoQuery	{ ClaseId: Int }	ConsultarAforoQueryHandler	Se salta el modelo de dominio y consulta directo a la BD (Dapper/SQL puro) devolviendo un DTO ligero. Alta velocidad.
Query: ReporteMensualQuery	{ Mes: Int, Anio: Int }	ReporteMensualQueryHandler	Previene el bloqueo de tablas transaccionales al leer millones de registros estadísticos.

6. Etapa 7: Análisis de Principios SOLID

La arquitectura propuesta ha sido escrutada contra los 5 principios SOLID del diseño orientado a objetos. A continuación, se detallan los hallazgos y evidencias:

1. Single Responsibility Principle (SRP): La clase 'ReservaApplicationService' tenía inicialmente lógica para calcular los recargos por fuera de plan. Hallazgo: Esto violaba el SRP. Solución: Se delegó el cálculo de recargos a un Value Object dentro de Membresía, dejando al servicio únicamente con la tarea de orquestar.

2. Open/Closed Principle (OCP): El sistema está abierto a la extensión y cerrado a la modificación. Evidencia: Si FitZone introduce una 'Membresía Diamante', no necesitamos tocar los servicios (if tipo == 'Diamante'). Simplemente creamos una nueva instancia del Value Object 'PlanContratado' con sus nuevas reglas y el sistema lo procesará polimórficamente.

3. Liskov Substitution Principle (LSP): Las interfaces de infraestructura cumplen con el contrato. Evidencia: Cualquier implementación de 'IClaseEspecialRepository' (ya sea en MySQL, SQL Server o Mock para Testing) devuelve la entidad con el mismo comportamiento esperado sin lanzar excepciones no controladas.

4. Interface Segregation Principle (ISP): No existe un 'IRepository' monolítico gigante. Evidencia: La interfaz 'IInstructorRepository' tiene solo 2 métodos pertinentes para el instructor, evitando que este módulo dependa de métodos de facturación o pagos que no necesita.

5. Dependency Inversion Principle (DIP): El núcleo del negocio no depende de los detalles de la base de datos. Evidencia: La capa 'Domain' no tiene referencias a Entity Framework, Dapper ni librerías de bases de datos. Todo el acceso a datos se inyecta desde la capa 'Infrastructure' mediante Inyección de Dependencias (IoC) en el arranque del sistema.

7. Etapa 8: Architecture Decision Records (ADRs)

ADR 0001: Implementación de Patrón Domain Model

Campo del ADR	Contenido
Título	Implementación de Patrón Domain Model
Estado	ACEPTADA
Contexto	FitZone presenta un nivel elevado de complejidad en la gestión de aforos físicos y reglas restrictivas basadas en la membresía. Utilizar Transaction Scripts generaría duplicación de lógica de validación en múltiples controladores.
Alternativas	Transaction Script (descartado por mantenibilidad), Table Module (descartado por acoplamiento a base de datos).
Decisión	Adoptar Domain Model como núcleo lógico.
Consecuencias	**Positivas:** Altísima cohesión, el código expresa el negocio (Lenguaje Ubicuo), protección total de invariantes. **Negativas:** Requiere que todos los desarrolladores del equipo comprendan los conceptos de DDD, lo que incrementa la curva de aprendizaje inicial.
Trazabilidad	US-01 (Acceso), US-06 (Suspendidas), US-07 (No-Shows)

ADR 0002: Separación Transaccional con Unit of Work

Campo del ADR	Contenido
Título	Separación Transaccional con Unit of Work
Estado	ACEPTADA
Contexto	Existen procesos como la 'Asignación automática desde Lista de Espera' que modifican múltiples entidades (Reserva, Lista de Espera, Notificaciones) simultáneamente.
Alternativas	Guardar los cambios repositorio por repositorio de forma aislada.
Decisión	Utilizar el Patrón Unit of Work para consolidar todos los commits de la base de datos en una sola transacción.
Consecuencias	Previene la corrupción de datos en caso de excepciones o caídas de red a mitad de un proceso. Centraliza el manejo del DbContext/Conexión.
Trazabilidad	US-02 (Reserva), US-04 (Cancelación), US-05 (Asignación Automática), US-12 (Cobro extra)

ADR 0003: Arquitectura CQRS para Consultas Masivas

Campo del ADR	Contenido
Título	Arquitectura CQRS para Consultas Masivas
Estado	ACEPTADA

Campo del ADR	Contenido
Contexto	El Administrador genera reportes mensuales de demanda (Lectura masiva). Cargar los Agregados de Dominio completos con todo su comportamiento para simplemente mostrar métricas colapsaría la memoria del servidor.
Alternativas	Crear repositorios de lectura convencionales que devuelvan entidades completas del dominio.
Decisión	Implementar CQRS. Los comandos (escritura) usan el Domain Model. Las consultas (lectura) atacan la base de datos con SQL puro o micro-ORMs (Dapper) y devuelven Data Transfer Objects (DTOs) planos.
Consecuencias	Escalabilidad asimétrica (podemos escalar la lectura independientemente de la escritura). Código más rápido pero con mayor cantidad de clases (Handlers, DTOs).
Trazabilidad	US-08 (Consulta de Aforo), US-10 (Reportes de Demanda)

8. Etapa 9: Bitácora de Inteligencia Artificial

Se utilizaron asistentes de Inteligencia Artificial para refinar y estructurar el diseño arquitectural, manteniendo siempre el escrutinio crítico y la responsabilidad sobre la decisión final.

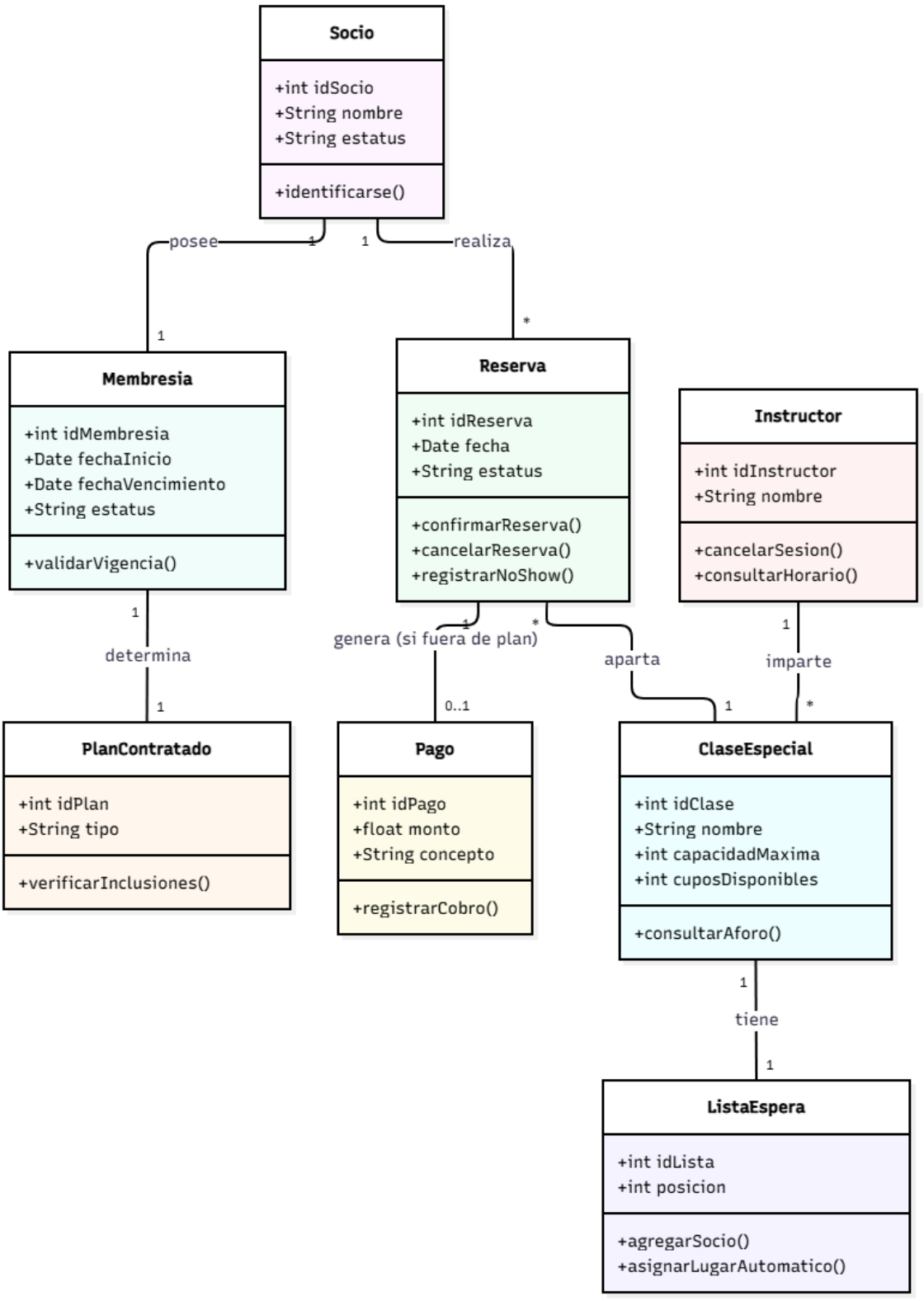
Objetivo del Prompt	Respuesta Útil de la IA	Error, Sesgo o Supuesto Detectado	Decisión y Corrección Humana
Generación de Tabla Comparativa de Patrones	Estructuró los criterios técnicos de manera clara y profesional.	Asumió que usaríamos Table Module basándose en bases de datos relacionales tradicionales.	Se descartó Table Module explícitamente porque no cumple con la necesidad de encapsular invariantes complejos del BRD.
Propuesta de Mapa de Dominio (Aggregates)	Identificó correctamente a Socio y Membresía como raíces del sistema.	Determinó que "Reserva" debía ser un Aggregate Root independiente.	Se analizó la cohesión y el equipo corrigió a la IA: Reserva no tiene sentido de existir sin una ClaseEspecial. Se movió Reserva a Entidad interna del Aggregate ClaseEspecial.
Traducción de Flujos a Código UML (Mermaid/PlantUML)	Generó toda la sintaxis de relación y clases sin errores de tipografía.	Omitió los bloques condicionales (alt/else) para representar rechazos de tarjetas o penalizaciones por No-Show.	El equipo obligó a la IA a reestructurar los diagramas de secuencia integrando lógicas de decisión (alt) requeridas en la Rúbrica.

9. Diagramas Arquitecturales y Modelado UML

En esta sección se integran las representaciones visuales generadas a partir del análisis y código PlantUML.

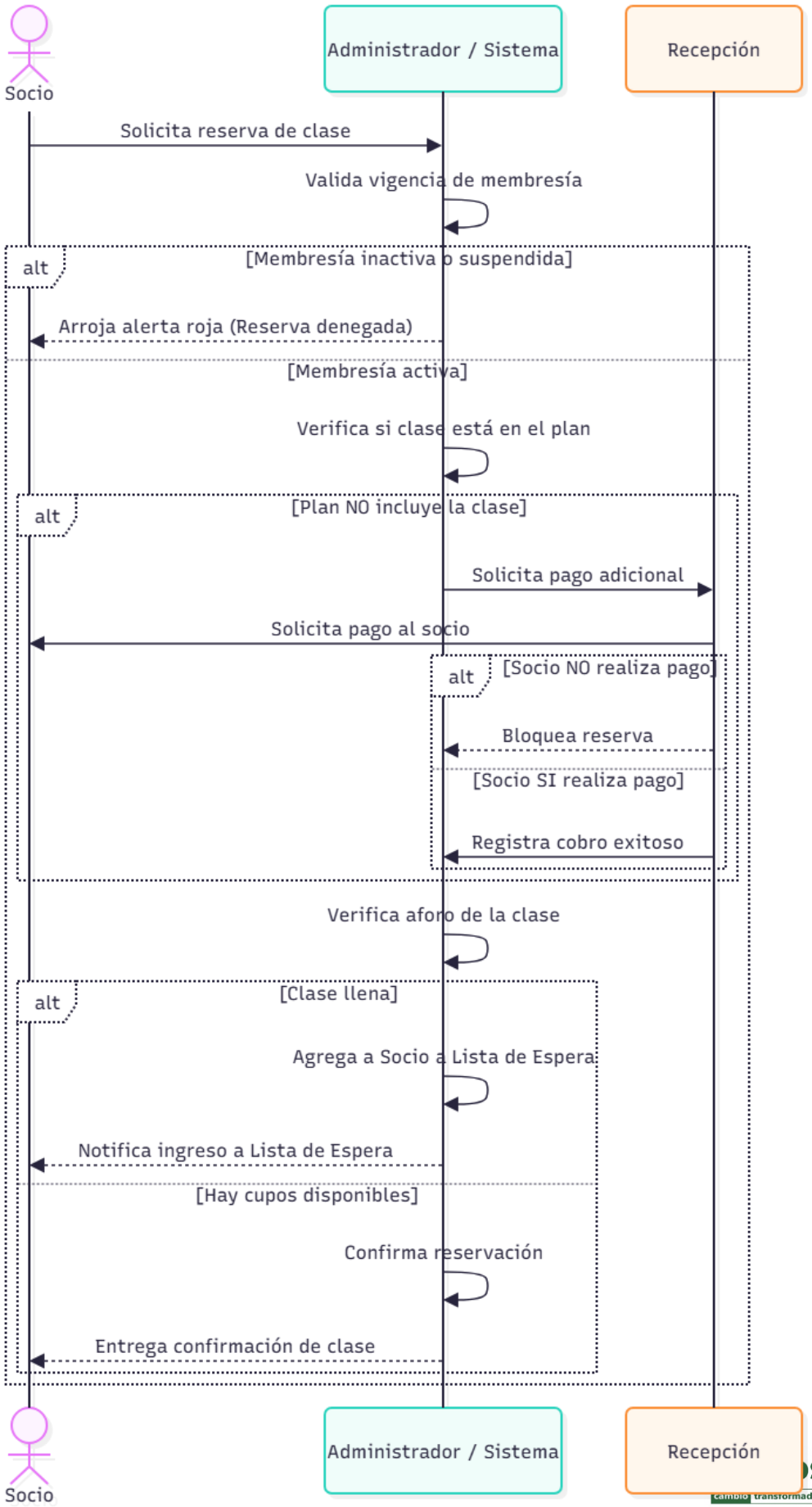
DIAGRAMA DE CLASES UML

Reporte


Socio
+int idSocio
+String nombre
+String estatus
+identificarse()
posee
1
1
realiza
1
*
Membresia
+int idMembresia
+Date fechaInicio
+Date fechaVencimiento
+String estatus
+validarVigencia()
Reserva
+int idReserva
+Date fecha
+String estatus
+confirmarReserva()
+cancelarReserva()
+registrarNoShow()
Instructor
+int idInstructor
+String nombre
+cancelarSesion()
+consultarHorario()
1
determina
1
1
genera (si fuera de plan)
0..1
*
aparta
1
1
imparte
*
PlanContratado
+int idPlan
+String tipo
+verificarInclusiones()
Pago
+int idPago
+float monto
+String concepto
+registrarCobro()
ClaseEspecial
+int idClase
+String nombre
+int capacidadMaxima
+int cuposDisponibles
+consultarAforo()
1
tiene
1
ListaEspera
+int idLista
+int posicion
+agregarSocio()
+asignarLugarAutomatico()

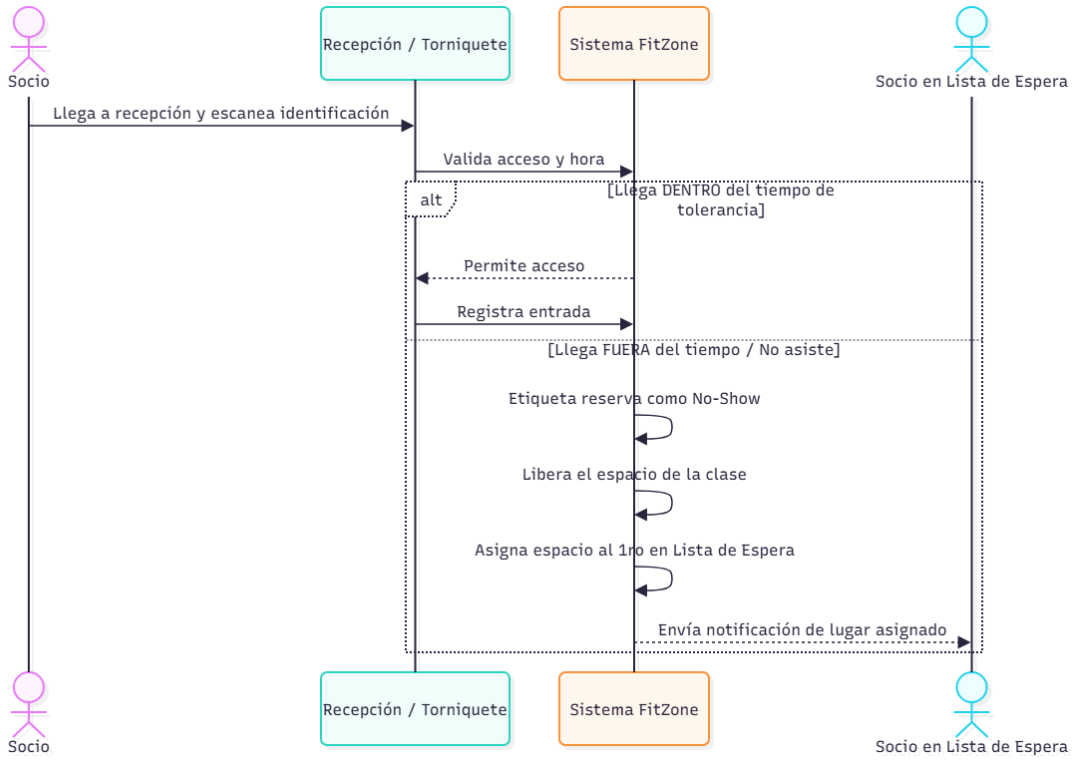
Reporte

DIAGRAMA DE SECUENCIA 1: FLUJO DE RESERVA Y COBROS


Socio
Administrador / Sistema
Recepción
Solicita reserva de clase
Valida vigencia de membresía
alt
[Membresía inactiva o suspendida]
Arroja alerta roja (Reserva denegada)
[Membresía activa]
Verifica si clase está en el plan
alt
[Plan NO incluye la clase]
Solicita pago adicional
Solicita pago al socio
alt
[Socio NO realiza pago]
Bloquea reserva
[Socio SI realiza pago]
Registra cobro exitoso
Verifica aforo de la clase
alt
[Clase llena]
Agrega a Socio a Lista de Espera
Notifica ingreso a Lista de Espera
[Hay cupos disponibles]
Confirma reservación
Entrega confirmación de clase
Socio
Administrador / Sistema
Recepción

Reporte

DIAGRAMA DE SECUENCIA 2: ACCESO, NO-SHOW Y LISTA DE ESPERA



Reporte

10. Matriz de trazabilidad

Matriz de Trazabilidad Arquitectural

Necesidad de Negocio (BRD)	Historias de Usuario (U2)	Caso de Uso (U3)	Componentes Arquitecturales (Dominio / Servicio)	Decisión Relacionada (ADR)
Control de Accesos: Validar que el socio tenga vigencia y estatus permitido.	US-01, US-06, US-14	CU-01: Identificación y Acceso en Recepción	Aggregate Membresia, AccesoAppService, IMembresiaRepository	**ADR 0001:** Domain Model para aislar la validación de vigencia.
Control de Aforo: No sobrepasar la capacidad máxima física del gimnasio.	US-02, US-04, US-08, US-15	CU-02: Reservación de Clase Específica	Aggregate ClaseEspecial, Entidad Reserva, ReservaAppService	**ADR 0002:** Unit of Work para garantizar atomicidad en los cupos.
Gestión de Lugares Ociosos: Reasignación automática de	US-03, US-05	CU-03: Ingreso a Lista de Espera	Entidad ListaEspera, Evento LugarLiberado, ListaEsperaAppService	**ADR 0001:** Uso de Eventos de Dominio para notificaciones.

Necesidad de Negocio (BRD)	**Historias de Usuario (U2)**	**Caso de Uso (U3)**	**Componentes Arquitecturales (Dominio / Servicio)**	**Decisión Relacionada (ADR)**
cancelaciones.				
Cobros Adicionales: Restricción de acceso a clases no incluidas en el plan.	US-12, US-13	CU-04: Cobro de Clases Fuera de Plan	Value Object PlanContratado, Entidad Pago, PagoAppService	**ADR 0002:** Transacción atómica (Pago + Reserva).
Manejo de Excepciones: Castigo operativo por reservar y no asistir.	US-07	*Flujo interno de validación de sistema*	Evento NoShowRegistrado, AsistenciaAppService	**ADR 0001:** Encapsulamiento de reglas de estado.
Métricas: Medir la demanda de las clases para futuros horarios.	US-10	*Flujo de lectura del administrador*	ReporteDemandaQuery, ReporteMensualQueryHandler	**ADR 0003:** Implementación de CQRS para lecturas pesadas.

12. Link de GitHub

https://github.com/DenilsonGonzalez07/FitZone/tree/main

13. Conclusiones

La transición desde el análisis de requerimientos (Unidad 2) hacia la definición de una arquitectura de software orientada al dominio (Unidad 3) ha permitido al equipo de desarrollo establecer una base sólida, predecible y altamente cohesionada para la construcción del sistema FitZone.

Al haber elegido Domain Model como nuestro patrón principal, aseguramos que el sistema será capaz de soportar la complejidad inherente de las reglas de negocio del gimnasio (control de aforos, cálculo de vigencias, bloqueos por suspensión y manejo dinámico de listas de espera) sin delegar dicha responsabilidad a la base de datos o a los controladores de interfaz de usuario. Esta centralización en 'Aggregates' (como Socio y ClaseEspecial) previene la corrupción del estado del sistema.

Complementariamente, la integración del patrón CQRS resolvió el conflicto clásico entre rendimiento y consistencia. Al separar los 'Commands' (operaciones críticas que mutan el estado y disparan eventos de dominio) de los 'Queries' (operaciones de lectura optimizadas con DTOs), garantizamos que la generación de reportes de demanda estadísticos no afecte el tiempo de respuesta del torniquete de recepción ni los bloqueos de base de datos durante las horas pico de acceso.

Finalmente, la rigurosa revisión a través de los principios SOLID y la documentación formal en Architecture Decision Records (ADRs) aseguran que el equipo cuenta con una guía técnica estandarizada. El código resultante será desacoplado, mantenible y abierto a la extensión, permitiendo que futuras implementaciones (como nuevos planes de membresía o integraciones con pasarelas de pago externas) se realicen con un impacto mínimo en la infraestructura existente, cumpliendo cabalmente con los criterios de éxito estipulados en el BRD original.